

Lecture 15

Composition and Aggregation

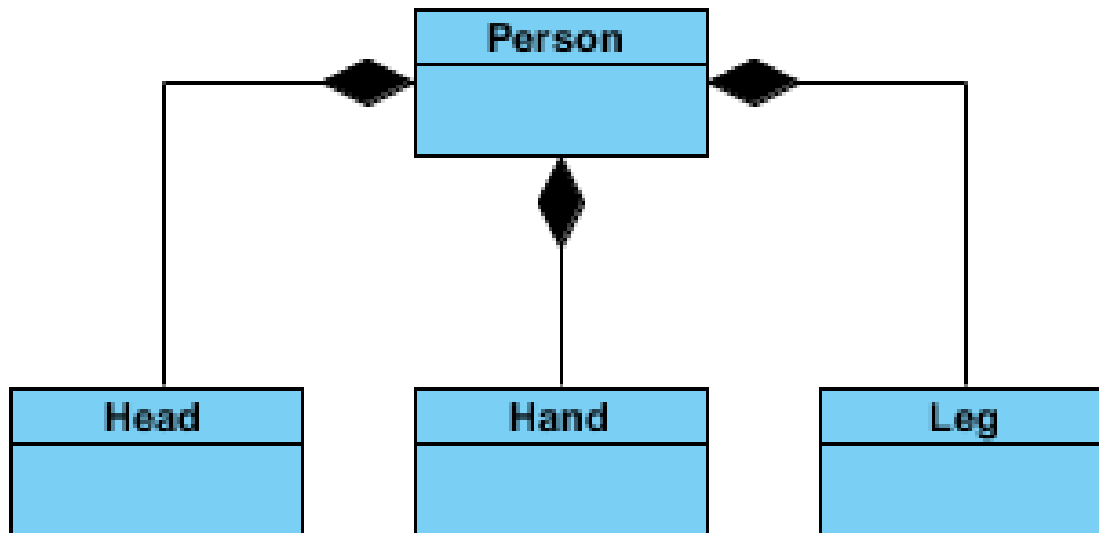
Object Oriented
Programming

Composition

- In real life objects are composed of multiple objects
- Example
 - Person *has* Legs, Head, Arms, etc.
 - Employee *has* birthdate, hiredate etc.
 - Car *has* tires, steering wheel, Windscreen etc.
 - House *has* rooms
- This phenomenon of Complex objects being composed of simpler one is known as ***composition***
- It is also known as **HAS-A** Relationship
- Unidirectional object of container class knows about its parts
- **Object composition** also allows for ***reusing functionality*** in object-oriented systems.

UML example for composition

- A Person has head, hands and legs etc.
- Each of them can be represented by a Class.
- However, objects Head, Hand and Leg do not exist without Person.
- When the Person class is created, the subclasses are created. When the Person class is destroyed, the subclasses are simultaneously destroyed.
- In UML, *composition* is depicted as a filled diamond and a solid line.



Composition (whole-part) implementation

- There are two classes A and B.
- Every object of A contains an object of class B.
- when an object of A is created object of B is created inside A too.
- When object of A is destroyed, object of class B is also destroyed.

```
class B {  
    int b;  
    Public:  
    B(int b=0) { this->b=b; }  
}  
  
Class A {  
    int a;  
    B objB;  
    public:  
    A(int a=0, int b=0):objB(b) {  
        this->a=a; }  
};
```

```
int main() {  
    A a; //obj a contains an object of  
        class B.  
    A a2(3,4); //whole as well as part is  
                created  
    return 0;  
}
```

Composition (whole-part) implementation

- There are two classes A and B.
- Every object of A contains a pointer to object of class B.
- when an object of A is created object of B is created inside A too.
- When object of A is destroyed, object of class B is also destroyed.

```
class B {  
    int b;  
    Public:  
    B(int b=0) { this->b=b; }  
}  
Class A {  
    int a;  
    B *objB;  
    public:  
    A(int a=0, int b=0): {  
        this->a=a;  
        objB = new B(b); }  
    ~A () { delete objB; }  
};
```

```
int main() {  
    A a; //obj a contains an object of  
        class B.  
    A a2(3,4); //whole as well as part is  
                created  
  
    return 0;  
}
```

Composition: An Example

- Many games and simulations have vehicles (such car and bike) that move around a board, map, or screen.
- One thing that all of these vehicles have in common is that they all have a location.
- We create a vehicle class that uses a 2Dpoint class to hold the vehicle's location.
- First we design a class named 2Dpoint.
 - The 2Dpoint class has 2 dimensions, X and Y.
 - And probably a constructor and one or few member functions.
- Class Vehicle has a few properties: a name, which will be a string, and a location, which will be an object of the Point2D class.
- We can read the composition as:
 - A vehicle **has-a** location
 - Location **is-part** of the vehicle



Composition: Example

```
class Point2D{
private:
int m_x;
int m_y;
public:
Point2D(int x,int y):m_x(x),m_y(y)
{
}
// Member functions
void setPoint(int x, int y){
    m_x = x;
    m_y = y;
}
};
```

```
class Vehicle{
private:
string m_name;
Point2D m_location;
public:
Vehicle(string name, int x, int y):
m_name( name ), m_location(x,y) {
}

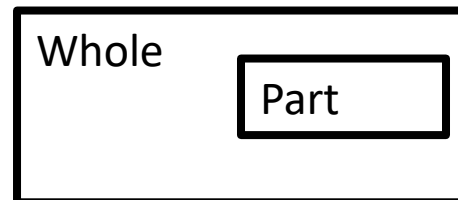
void moveTo(int x, int y) {
m_location.setPoint(x, y);
}
};
```

```
int main(){

Vehicle Vehicle("Benz",4,7);
return 0;
}
```

Composition

- Some characteristics of the composition:
 - **Lifetime:** The two objects are created and destroyed at the same time. It also means that the relationship between the two objects is permanent - it cannot be altered or broken.
 - Composed object (part) cannot live without the composing object (whole).
 - For example, `birthdate` object (of class `Date`) without the object of class `Employee` is meaningless.
 - Composition binds the part objects strongly or tightly to the whole object - the compiler embeds the parts inside the whole. the whole object does not share its part object with any other object in the program.



Composition: Another Example

- Imagine there is an employee class which among other necessary data members and functions (name, salary etc.) also has an embedded object of Class account.
- Class Account has only two data members, accountID and initBalance as well as member function display().
- Class Employee has two data members name, salary and account as well as a member function Edisplay().
- Every time, an Employee object is created an embedded Account object is also created.



```

class account {
int accountID, initBalance;
public:
account(int ID, int Bal) : accountID(ID), initBalance(Bal) { }
void display() {
cout << " Account ID: " << accountID << " and balance: " << initBalance << endl;
} };

```

```

class Employee {
string name;
double salary;
account ac;
public:
Employee(string n, double sal, int id, int bal) : ac(id,bal), name(n),salary(sal) {}
void Edisplay() {
cout << "Name: " << name << " Salary: " << salary << endl;
ac.display();
}
};

```

```

Name: Gareth Thomas Salary: 200000
Account ID: 139 and balance: 250000

```

```

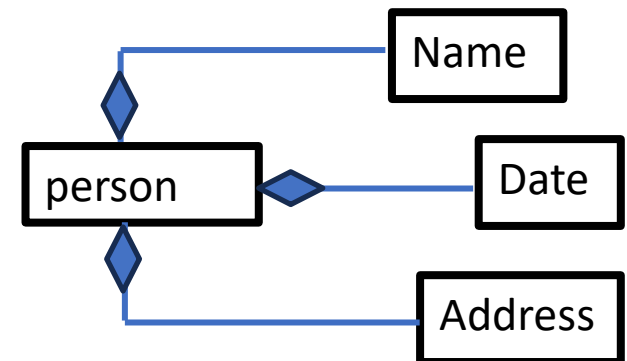
int main() {
Employee E1("Gareth Thomas", 200000,139, 250000);
E1.display();
}

```

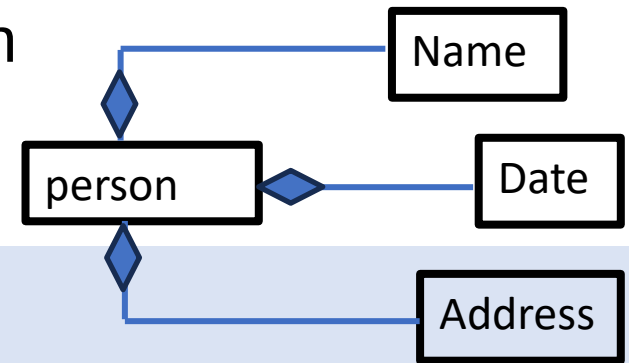
Separating a class in different subclasses for composition

- Suppose we have a class named person. So instead of putting all related information in the class and making the class control it, it is better to make separate classes such as date , address and make them part of the person class.

```
class person {  
    string name;  
    //date information  
    int day, month, year;  
  
    //address information  
    string city,  
    String country;  
    int strno, houseno;  
    //person specific info  
    int id;  
  
    //constructor  
};
```



Composition (whole-part) implementation



```
class name {
string _name;
public: name(string n): _name(n){}
};

class date {
int day, month, year;
public:      date(int d, int m, int y): day(d), month(m),
year(y) {}
};

class address {
string city, country;
int strno, houseno;
public:
address(string c, string cntry, int stno, int hno): city(c),
country(cntry), strno(stno),houseno(hno) {}
};
```

Composition (whole-part) implementation

```
class person {
int id;
name *pname;
date *dateofbirth;
address *paddress;
public:
person(int pid, string n, int d, int m, int y, string city,
string country, int street, int house ) {
    dateofbirth  = new date(d,m,y);
    paddress = new address(city, country, street, house);
    pname = new name(n);
    id = pid;}
};
```

```
date constructor:
address constructor:
name constructor:
person constructor:
*****
```

```
int main() {
person p (1, "kangWoo Lee", 28, 3, 72, "Seoul", "South Korea", 20, 76);
}
```

Composition (whole-part) implementation

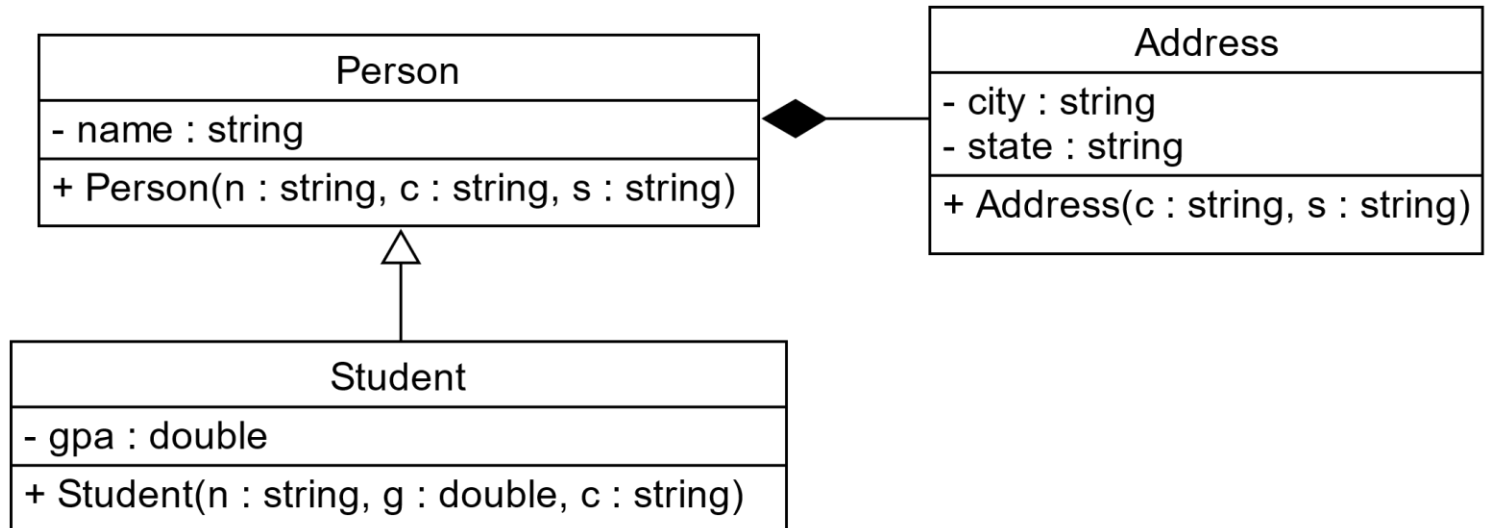
```
class name {...
public:
~name(){cout<<"name destructor: "<<endl;}};
class date {...
public: ...
~date() {cout<<"date destructor: "<<endl;}};
class address {...
public: ...
~address(){cout<<"address destructor: "<<endl;}};
class person {...
public:...
~person() { // call destructors of composed objects within
//destructor of person, sequence is controlled by programmer
delete pname;
delete dateofbirth;
delete paddress;
cout<<"person destructor: "<<endl;}
};
```

```
date constructor:
address constructor:
name constructor:
person constructor:
*****
name destructor:
date destructor:
address destructor:
person destructor:
```

```
int main() {
person p (1, "kangWoo Lee", 28, 3, 72, "Seoul", "South Korea", 20, 76);}
}
```

Inheritance and Composition

- Classes may participate in multiple relationships at the same time.
- The following three classes (Person, Address, and Student) connected by two relationships.
 - Inheritance
 - Composition
- Class Person has two roles:
 - **Inheritance:** A student is-a Person
 - **Composition:** A person has-an address.



Inheritance and Composition: Example

```
class Address {
    private:
        string city;
        string state;
    public:
        Address(string c, string s) : city(c), state(s) {}
};

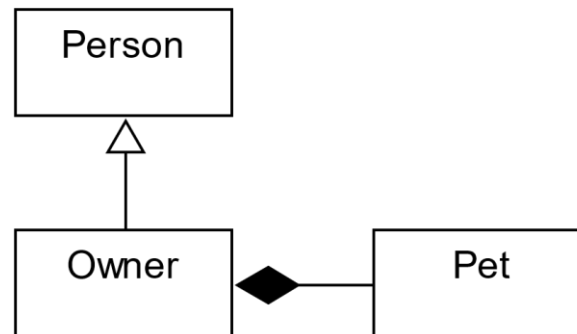
class Person
    private:
        string name;
        Address addr;
    public:
        Person(string n, string c, string s) : addr(c, s), name(n)
{}
};

class Student: public Person    {
    private:
        double gpa;
    public:
        Student(string n, double g, string c, string s)
            : Person(n, c, s), gpa(g) {}
}

int main() {
    Student S("Ahmed", 3.14, "Lahore", "Punjab");
}
```

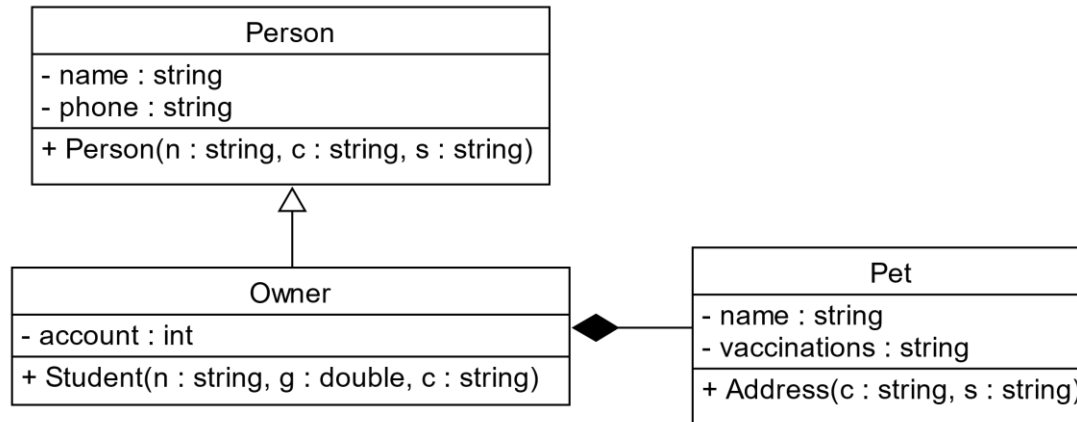
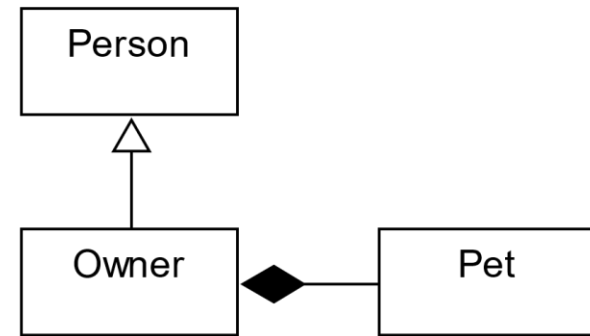

Inheritance and Composition

- Classes may participate in multiple relationships at the same time.
- Following three classes connected by two relationships.
 - Inheritance
 - Composition
- Now Owner plays two roles:
 - **Inheritance:** Person (subclass) is an Owner (superclass)
 - **whole-part:** Owner (whole) has a Pet (part)



Inheritance and Composition

- Classes may participate in multiple relationships at the same time.
- Following three classes connected by two relationships.
 - Inheritance
 - Composition
- Now Owner plays two roles:
 - **Inheritance:** Person (subclass) is an Owner (superclass)
 - **whole-part:** Owner (whole) has a Pet (part)



Composition(one to many)

- The relationship between a house and rooms is composition. Each house consists of several rooms. Each house have some number of rooms, each room having its unique id and size in feet.

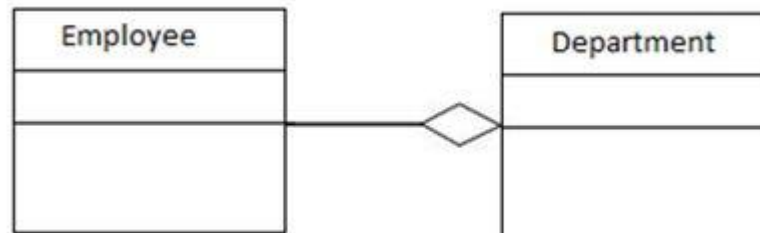
```
class Room {  
    int rid;  
    float size;  
public:  
    Room(int t=0, float s=0.0) {  
        rid=t;  
        size = s; }  
};
```

```
int main(){  
    House h1(1);  
}
```

```
class House {  
    int hid;  
    int noofrooms;  
    Room * rlist[10];  
public:  
    House(int i, int rooms){  
        for(int j=0;j<10;j++)  
            rlist[j]=new Room(j+1, j+10*j+5);  
    }  
    ~House() {  
        for(int i=0;i<10;i++)  
            delete rlist[i];  
    }  
};
```

Aggregation

- It is a weak **HAS-A** relationship
- The composed object (part) can live without the composing object (whole)
- Examples:
 - Department **has-a** Teacher
 - Room **has-a** Chair
 - Employee has-an officeAddress
- **Aggregation** is attained by creating reference or pointer to the composed object in the composing object.
- The UML forms the **aggregation connector** as a line with a hollow or outlined diamond attached to the whole class.



Aggregation

- Some characteristics of the **Aggregation**:
 - Implements the composed object (part) as pointer member variables in the composing object (whole class)
 - Binds the part objects weakly or loosely to the whole object - the parts exist outside the whole
 - The parts and the whole may be created and destroyed at different times
 - The parts can be changed or replaced whenever it is convenient

Aggregation: Example

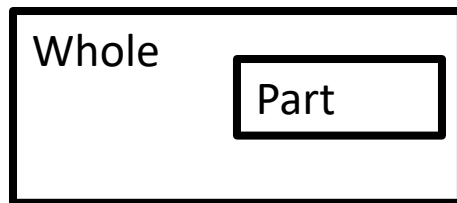
```
class Teacher{
private:
string m_name;
int m_tid;
String m_subject;
public:
Teacher(string name, int tid,
string subject): m_name(name),
m_tid(tid), m_subject(subject)
{ }
string getName() {
    return m_name; }
};
```

```
class Department{
private:
Teacher *m_teacher;
public:
Department(Teacher *teacher):
m_teacher( teacher)  { }
};
```

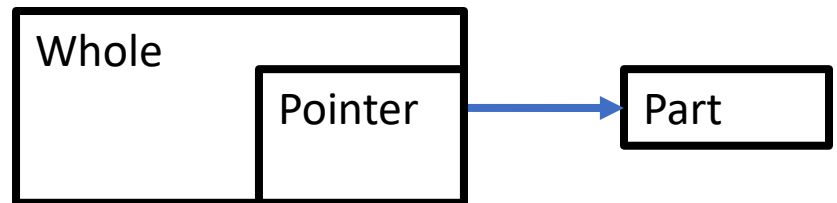
```
int main(){
Teacher T1("Haris", 210, "Data Science");
Department department(&T1);
cout << T1.getName();
}
```

Composition vs aggregation

- **Composition:** Binds the part objects strongly or tightly to the whole object - the compiler embeds the parts inside the whole.
- **Aggregation:** a pointer member variable in the whole object stores the address of the part.



Composition



Aggregation